

Lesson 1: Streaming Video from a USB Camera

Your ROV is sealed inside its hull underwater — you can't see it. The camera is how you see what it sees. In this lesson you'll get a live video feed from a USB camera on the Raspberry Pi showing up in a browser on your laptop, over the network.

What you'll build

A small Python program that:

1. Reads frames from the USB camera using **OpenCV**
2. Compresses each frame to a JPEG image
3. Runs a tiny web server that sends a continuous stream of those JPEGs to any browser that connects — a format called **MJPEG** (Motion JPEG)

When you're done, opening `http://<pi-ip-address>:8000/` in a browser on your laptop will show the live camera feed, updating continuously, with the Pi doing all the work and your laptop just displaying the result.

Why stream over the network instead of showing it on the Pi?

The Pi is going to be sealed inside the ROV with no monitor attached. The only way to see its camera is to send the video *off* the Pi, over the tether/network, to a laptop at the surface. This is exactly how real ROVs work — everything the pilot sees comes over a network or tether connection, not from a screen on the vehicle itself.

Background: how a digital camera becomes a browser-friendly video

1. **The camera hardware** shows up on Linux as a device file, typically `/dev/video0` (or `/dev/video1`, etc. if

you have more than one camera). Index `0` is what you'll pass to OpenCV to say "use this camera."

2. **OpenCV's** `cv2.VideoCapture` talks to that device and gives you frames — each one is just a grid of pixel values (a NumPy array), the same as a photo.
3. **JPEG compression** (`cv2.imencode`) squeezes each frame down to a much smaller size. Raw video is too large to send efficiently; JPEG trades a little image quality for a huge reduction in data size, which matters a lot on a network link.
4. **MJPEG over HTTP** is a trick that's been around for decades: instead of a video *file*, the server keeps one HTTP response open forever and keeps writing new JPEG images into it, one after another, each separated by a marker called a **boundary**. A browser showing that response in an `<img>` tag just keeps redrawing the image as new ones arrive — which looks exactly like video.

Reading the code:

`stream_server.py`

Open `stream_server.py` in PyCharm. Here's what each piece does.

Camera — grabbing frames in the background

```
self._thread = threading.Thread(target=self._capture
```

Reading from a camera can momentarily block while waiting for the next frame. If we only read a frame when a browser asks for one, every viewer would be stuck waiting on the camera hardware. Instead, a **background thread** continuously reads frames and stores only the most recent one. When a browser asks for a frame, it instantly gets whatever's already there — no waiting on the camera.

A `threading.Lock` guards the shared `_latest_jpeg` variable so the capture thread and the web server thread never read/write it at the exact same instant and corrupt it.

StreamingHandler — the web server side

This is built on Python's standard `http.server`. Two URLs are handled:

- `/` returns a tiny HTML page containing `` — this is the page your browser actually opens.
- `/stream.mjpg` is the actual video stream. Look at `_serve_stream` :

```
self.send_header("Content-Type", f"multipart/x-mixed-replace; boundary={boundary}")
```

This HTTP header is what tells the browser "I'm not sending you one image, I'm sending you a never-ending sequence of images — keep displaying each new one as it arrives." Then the loop just keeps writing `boundary + headers + one JPEG` over and over, forever, until the browser disconnects.

ThreadingHTTPServer

A plain `HTTPServer` can only handle one connection at a time. Since the video stream holds its connection open indefinitely, a second viewer (or even just reloading the page) would hang forever waiting for the first connection to finish. `ThreadingHTTPServer` spins up a new thread per connection so multiple people can view the stream at once.

`main()`

Parses command-line options (camera index, port, resolution), starts the camera, starts the server, and on `Ctrl+C` shuts both down cleanly so the camera device is released properly.

Running it

On the Pi (see the [shell lesson](#) if any of this is unfamiliar):

```
► cd robotics-lesson/camera
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python3 stream_server.py
```

You should see:

```
Streaming at http://<pi-ip-address>:8000/ (Ctrl+C to stop)
```

Now open that address in a browser **on your laptop** (not the Pi). You should see the live camera feed. Press `Ctrl+C` in the terminal to stop the server.

If it doesn't work

Problem	Likely cause
Could not open camera 0	Camera isn't plugged in, or it's <code>/dev/video1</code> instead — try <code>--camera 1</code>
Permission denied opening	Your user isn't in the <code>video</code> group — see the shell lesson, section 5

Problem	Likely cause
the camera	
Page loads but never shows an image	Check the Pi's firewall isn't blocking port 8000; confirm you used the Pi's IP, not <code>localhost</code> , from your laptop
Laptop can't reach the Pi's IP at all	Confirm both devices are on the same network; re-check <code>hostname -I</code> on the Pi

Measuring real-world latency with a phone

Latency is the delay between something happening in front of the camera and you actually seeing it on your laptop screen — capture, JPEG encoding, network transfer, and browser decoding all take a small amount of time. For piloting an ROV, high latency is the difference between "I see a wall and stop in time" and "I see a wall after I've already hit it," so it's worth measuring, not just assuming it's small.

You can't measure this with a regular stopwatch, because by the time you read the stopwatch and compare it to the

screen, you've introduced your own reaction-time error. The trick is to capture *both* the real time and the delayed time **in a single photo**, using a second phone.

What you need

- The ROV's USB camera, running `stream_server.py` , viewed in a browser on your laptop
- A phone showing a **millisecond stopwatch** (many free stopwatch apps show `MM:SS.mmm` ; a web search for "online millisecond stopwatch" also works in a mobile browser)
- A second phone (or anyone's phone camera) to take the measurement photo

Steps

1. Start the millisecond stopwatch on the first phone and let it run continuously.
2. Point the USB camera directly at that phone's screen, close enough that the numbers are sharp and readable in the stream.
3. On your laptop, open the stream (`http://<pi-ip-address>:8000/`) so it's clearly visible — you're now looking at a video of a stopwatch, running slightly behind the real one.

4. Using the **second phone**, take a single photo that frames **both** screens at once: the original stopwatch phone, and the laptop screen showing the stream. Both timestamps need to be sharp enough to read in the photo.
5. Zoom into the photo and read both timestamps. The real stopwatch will read a later (bigger) time than the one visible in the stream on the laptop — that difference **is** your end-to-end latency, in milliseconds.

Why this works

Both numbers exist at the exact same instant: the instant the measurement photo's shutter opens. One of them traveled straight to your eye at the speed of light (basically instant); the other traveled through the entire camera → Pi → network → browser pipeline you just built, which takes measurably longer. The photo freezes both of them side by side so you can read the gap directly, instead of trying to perceive it in real time.

Things to try

- Take several photos and average the readings — a single measurement can be off by a frame or two.

- Repeat at your original resolution and again at the smaller `320x240` resolution from the exercises below. Lower resolution means less data to encode and send per frame, so you should measure *lower* latency — this is the same tradeoff you noticed in smoothness, now with a number attached to it.
- Try it on a slow/congested Wi-Fi network vs. a direct wired connection to see the network's contribution to the delay.

Exercises

1. Run the server and view the stream from your laptop's browser.
2. Change `--width / --height` to a smaller resolution (e.g. `320x240`) and notice how much smoother the stream looks on a slow network — this is the same bandwidth-vs-quality tradeoff real ROV pilots deal with.
3. Open the stream in two browser tabs at once and confirm both update — this only works because of `ThreadingHTTPServer` . Try temporarily swapping it for a plain `HTTPServer` (same import line, different class name) and see what happens with two tabs open.
4. Add a `--fps` command-line option that controls the `time.sleep(1 / 30)` value instead of hard-coding

30.

5. Measure the stream's end-to-end latency using the phone-photo method above, at two different resolutions, and record both numbers.